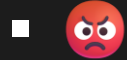


TILLÄMPAD ARTIFICIELL INTELLIGENS

F4 - World Interfacing

LITE FEEDBACK PÅ KOD

- auto



- Använd bara auto om det ökar läsbarheten.
 - Lätt att tänka att det blir enklare att läsa för en själv, men tänk att någon annan hade läst koden! Hade det varit enklare eller svårare för dem?
 - Ofta gör auto kod väldigt diffus/otydlig
 - Man måste steppa runt in och ut ur funktioner för att ta reda på typer.
 - Ibland kan man inte det! T.ex. om man review:ar kod i swarm.
 - I mitt team på Massive fick auto bara användas för iterators
- auto kan också strula till det för er ibland på andra sätt
 - T.ex. om ni gör `auto object = GetObject()` och `GetObject()` returnerar `Object&`.
 - Ni förväntar er att auto blir `Object&`, men det är upp till kompilatorn!
 - Ni kan få en unintentional copy.
- Av-vänj er vid auto asap
 - Ni måste förmodligen göra det förr eller senare 

LITE MER FEEDBACK PÅ KOD

- smart pointers
 - 🙄
 - Men ibland: 😊
 - Varför används smart pointers?
- Om ni inte vill enforce:a ägandeskap, använd raw pointers istället.
 - Jag brukar kommunicera ägandeskap utan smart pointers såhär:
 - `Type myThing; // myThing ägs av denna klassen`
 - `Type* myThing; // myThing skapas utanför denna klassen, men ägs fortfarande av den`
 - `const Type* myThing; // myThing skapas utanför denna klassen, och ägs inte av den`
- Om ni använder smart pointers utan en klar intention så blir ni bara sämre på att använda raw pointers.
- Smart pointers kan vara nice, men det abstraherar också bort viktig kunskap.
- Mitt tips
 - Om ni vill använda smart pointers, koda era egna klasser!
 - `CU::SharedPtr`, `CU::WeakPtr`, `CU::UniquePtr`
 - Annars använd raw pointers
 - Först blir vi raw pointer ninjor
 - Sen kan vi använda smart pointers
 - Om saker sprängs när vi använder raw pointers, nice! Då lär vi oss något om resurshantering.

SISTA FEEDBACKEN PÅ KOD

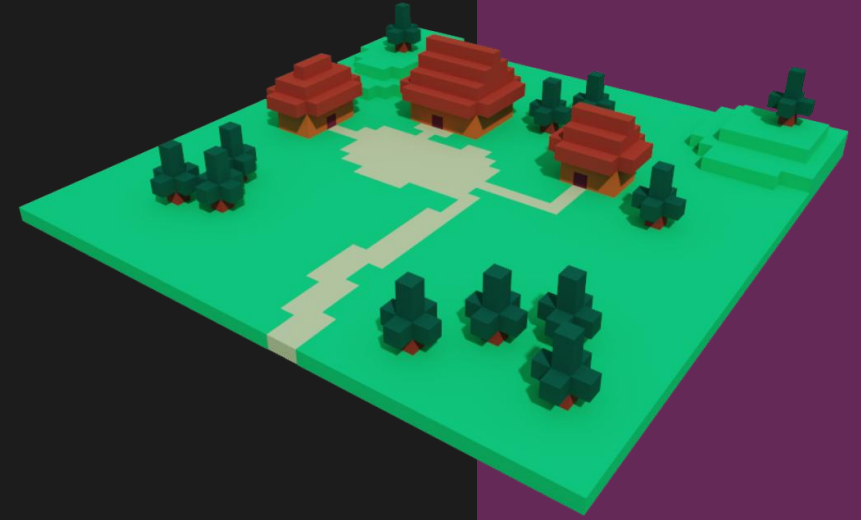
- Många av er definierar "boid" i kod.
 - Dvs som en ny class/type
 - Som variabelnamn, ok 😊
 - Som class/type 😡
- "Boid" är något som definieras av tre controllers/behaviours.
- Det kraftfulla är att vi kan skapa en "boid" genom att kombinera tre befintliga behaviours, utan att behöva skapa något som specifikt är en "boid".
- Det är ofta nice om vi kan fokusera på beteenden och funktion i kod.
 - Spoiler till U07:
 - Där jobbar vi med predators och preys
 - Vi kanske ger actors en wolf/sheep-sprite, men vi döper inget till wolf/sheep i kod. Vi håller oss till predator/prey.

INTRO

- En AI gör saker beroende på datan den får.
 - Är spelaren nära?
 - Är spelaren i line of sight?
 - Är det tillräckligt med fiender i vår level?
 - Är fienden skadad och behöver gå till ett health pack?
- På det stora hela kan vi säga att AI:n
 - Får input (stimuli)
 - Returnerar output (beteende/beslut)
- På något sätt måste vår AI få tillgång till data i spelvärlden.
 - Hur "interface:ar" AI:n med spelvärlden?
 - Detta kallas world interfacing!
 - Hittills har vi bara hämtat datan precis där den finns.
 - Kanske gjort GameWorld till en singleton? (jag har det, no shame)
 - Men håller det i längden? 🤔

WORLD DATA

- Att få tillgång till data kanske låter enkelt?
 - `GameWorld::GetInstance().GetPlayerPosition();`
 - Done
 - ... nja, finns lite saker vi bör tänka på.
- För små små spel kanske det där funkar.
 - Men vi vill bygga lösningar som går att skala upp!
- AI blir snabbt en stor del i en motor.
 - Värt att bygga klasser/managers/komponenter specifikt för AI.
 - Blir enklare att debugga och optimera just AI:n. Varför det är så kommer bli tydligare alldeles strax :)
- Finns huvudsakligen två sätt för en AI att få data
 - Polling
 - Events
- Vi börjar med att kolla på polling!



POLLING

- Att "poll:a" innebär att be om data.
- Varje frame säger vi "Ge mig spelarens position".
 - Ganska straight forward!
 - Dock bäst att göra det via en s.k. PollingStation.
- Vad är fördelarna med en PollingStation?
 1. Vi får en överblick av all data AI:n vill polla.
 2. När vi har get:at datan en gång så cache:ar vi den. Dvs om vi gettar datan flera gånger på en frame så beräknar vi den bara en gång.
- Vad är nackdelarna med en PollingStation?
 - Onödigt att konstant fråga efter data som inte ändrar på sig.

```
class AIPollingStation
{
public:
    > static AIPollingStation& GetInstance();

    > u32 GetPlayerHealth();
    > u32 GetNumberOfAliveEnemies();

private:
    > AIPollingStation();

    > template<typename T>
    > struct PollData
    > {
    >     > u32 myLastUpdatedFrame = 0;
    >     > T myData;
    > };

    > PollData<u32> myPlayerHealth;
    > PollData<u32> myNumberOfAliveEnemies;
};
```

POLLING

- Hur cachar vi datan i våra getters så den inte beräknas mer än en gång per frame?
 - Vi kan använda en frame count!
 - Easy peasy

```
u32 AIPollingStation::GetPlayerHealth()
{
    const u32 frameCount = Tga::Engine::GetInstance()->GetFrameCount();
    if (myPlayerHealth.myLastUpdatedFrame < frameCount)
    {
        myPlayerHealth.myLastUpdatedFrame = frameCount;
        // Interface with GameWorld or other systems to dig out
        // the data needed to get or calculate player health.
        // Set myPlayerHealth.myData to that value.
    }

    return myPlayerHealth.myData;
}
```

- Men kommer vi inte få overflow på vår frame count...?
 - Anta 60fps
 - u32 max är $2^{32} - 1$
 - $(2^{32} - 1) / 60 / 60 / 60 / 24 / 365 \approx 2.27$ år
 - Använder vi u64 får vi 9.8 miljarder år 🤖

EN STÖRIG GREJ MED POLLING

- Tänk er att vi gör GTA och vill ha `GetHasAnyBankBeenRobbed` i vår polling station.
 - Vi skulle kunna expose:a en lista med alla banker i vår game world.
 - Iterera genom den, returnera true om någon `myHasBeenRobbed` är true.
 - Men om class `Bank` inte har en `myHasBeenRobbed`?
 - Ska vi lägga till den då och manage:a det bara för att polling stationen vill ha den?
 - Eller ska vi lägga till `myHasAnyBankBeenRobbed` i game world?
 - AI-kod börjar leak:a in i all annan kod! Gah!!
- Ibland kommer man inte undan detta.
 - Särskilt inte i så små applikationer som kursens uppgifter.
 - Kan hända att ni t.ex. behöver lägga till `myHasAnyBankBeenRobbed` i er polling station och i game world för AI:n.
 - Sånt man får leva med ibland.
 - Om AI:n är intresserad av det är sannolikheten stor att även andra system vill nå samma data ändå.
- (Dagens bonus-råd)
 - Man ska inte nöja sig med mediokert shit.
 - Man ska heller inte låta perfektion komma i vägen för något tillräckligt bra.
 - Det är en balansgång, och nåt av det viktigaste att lära sig som spelutvecklare.

EVENTS

- Events innebär att vi chillar tills någon säger till oss att något har hänt eller att data har ändrat på sig.
- Ni är vana vid en PostMaster. En EventManager är som "en PostMaster för vår AI".
 - Varför inte använda samma?
 - PostMastern kan snabbt bli bloat:ad med massa AI-specifika events.
 - Enklare att "hålla koll" på just AI:n.
- Vad är fördelarna med en EventManager?
 1. Data som sällan ändrar på sig kommer inte beräknas varje frame.
 2. Events är nice att jobba med. Vi kan enkelt logga dem, och t.ex. skicka dem via key presses för enkel debugging.
- Vad är nackdelarna med en EventManager?
 - Extra overhead för event management.

```
template<typename EventEnumType>
class HD_EventManager
{
public:
    template<typename ListenerType, typename EventChildType>
    void RegisterEventListener(ListenerType* aListener, const EventReceivedCallback<ListenerType, EventChildType>& aEventReceivedCallback);

    void UnregisterEventListener(void* aListener);

    template<typename EventChildType>
    void SendEvent(const EventChildType& aEvent);

private:
    HD_HashMap<EventEnumType, HD_GrowingArray<HD_EventListener>> myEventListeners;
};

GameWorld::GetInstance().GetAIEventManager()->RegisterEventListener(this, &SmartGuardEventController::HandlePlayerStartedHackingMessage);
GameWorld::GetInstance().GetAIEventManager()->UnregisterEventListener(this);
```

(koden finns på Omniway)

POLLING VS EVENTS

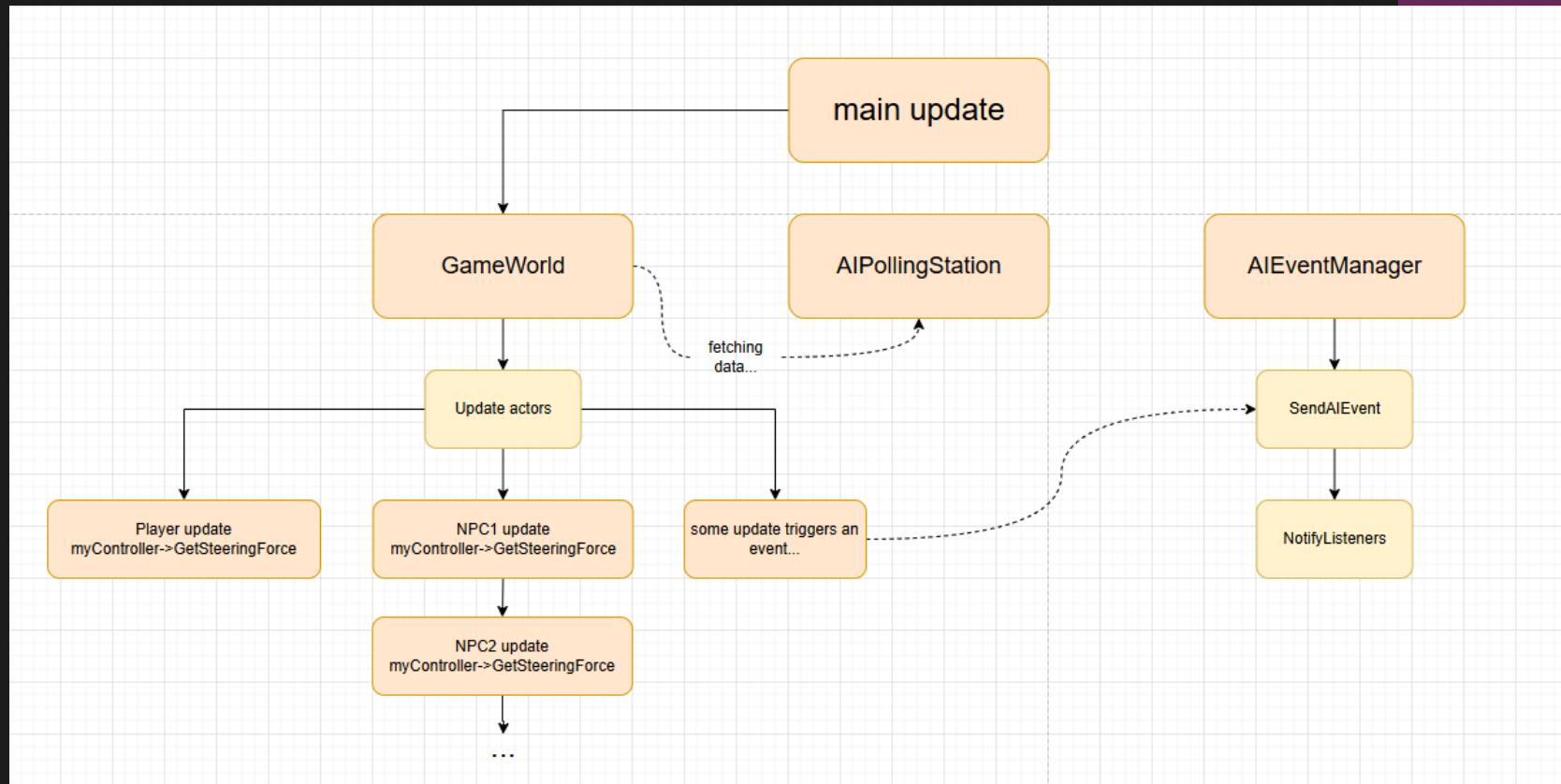
- Polling: "Är du redo nu?"
- Events: "Ring mig när du är redo."
- Kanske enkelt att tro att events alltid är vår goto, och att events är bättre.
 - Viktigt att komma ihåg att varken polling eller events är snabbare!
 - Beror helt på hur de används. Underskatta inte polling!
- Bottom line:
 - Om data används oftare än den ändras, använd events.
 - Om data ändras oftare än den används, använd polling.
- En vanlig lösning är att ändå använda events för allt.
 - Just eftersom de är nice att ha att göra med när det kommer till logging och debugging.
 - Och sen använder man en PollingStation för de bitarna som blir tramsiga att använda events för.
 - Antingen att events i vissa situationer bara blir för osmidigt, eller om det finns vissa kritiska delar i AI-koden som vi vill optimera genom polling.

POLLING/EVENTS - SAMMANFATTNING

- För att kunna bygga system för AI som
 - går att användas av flera olika spel,
 - gör det enkelt att ge rätt data till rätt delar av AI-kod,
 - ... i rätt format,
 - ... vid rätt tidpunkt,
 - så måste vi enkapsulera vår AI-kod. T.ex i dedikerade managers.
- Faktumet att all vår AI-kod flödar genom samma system gör det enklare att debugga, logga och optimera.
- Det hade det inte varit om vi interface:ade med GameWorld!

SAMMANFATTNINGSVIS

- PollingStation, EventManager, Actors, Controllers...
hur hänger allt det där ihop då?
- Typ såhär:



LITE NYTT OM CONTROLLERS

- Hittills har vi bara jobbat med controllers som är steering behaviours.
 - Sen la vi till så actors kan ha flera controllers, dvs vi kan kombinera flera steering behaviours.
 - Men det finns ju inget som egentligen hindrar oss från att ha helt customized controllers.
 - PlayerController
 - ProgrammingTeacherController
 - SillyMerchantController
 - UnhingedMilitaryGeneralController
 - Eller vad som helst som vårt spel behöver!
- I dessa controllers kan vi:
 - Mixa och matcha steering behaviours
 - "Hacka" versioner av existerande steering behaviours
 - Återigen, vad vi än behöver.
- Kom ihåg
 - Steering behaviours är inte enda sättet att styra movement
 - Vi kanske inte vill att vår spelare ska "begränsas" av steering behaviours
 - Då kan spelaren såklart vara en Player class istället för en Actor class
 - Kanske uppenbart... men lätt att glömma när man gräver ner sig i steering

KURSLITTERATUREN

- Denna föreläsningen
 - Chapter 11 - World Interfacing
 - 11.1 Communication (s 836-838)
 - 11.2 Event Managers (s 838-846)
 - 11.3 Polling Stations (s 846-849)
 - Extra:
 - 11.4 Sense Management (s 849-874)
- Nästa föreläsning
 - Chapter 4 - Pathfinding
 - 4.1 The Pathfinding Graph (s 195-203)
 - 4.2 Dijkstra (s 203-214)
 - 4.3 A* (s 215-237)
 - 4.4 World Representations (s 237-256)

UPPGIFTEN

- Vi ska arbeta med dataflöde (world interfacing) till AI controllers.
- När spelet börjar ska man se fem actors och ytterliggare tre statiska entities.
 - En actor är spelaren. Den ska styras av en PlayerController eller ArriveController som reagerar på mouse input. Spelaren ska röra sig dit man senast klickade.
 - Våra fyra andra actors är robotvakter som ska upptäcka när spelaren hackar datorer.
 - Våra tre statiska entities är datorer som spelaren kan hacka. Spelaren "hackar" dem genom att bara befinna sig inom ett visst avstånd från dem.
- Det finns två "smarta" vakter och två "dumma" vakter.
 - De smarta vakterna rör sig mot den senaste datorn som spelaren har försökt hacka och stannar inte förrän de är framme vid den, oavsett om spelaren fortfarande är där eller inte.
 - De dumma vakterna rör sig bara mot den datorn som spelaren just nu hackar. Om spelaren inte hackar någon dator just nu så står de stilla.

UPPGIFTEN

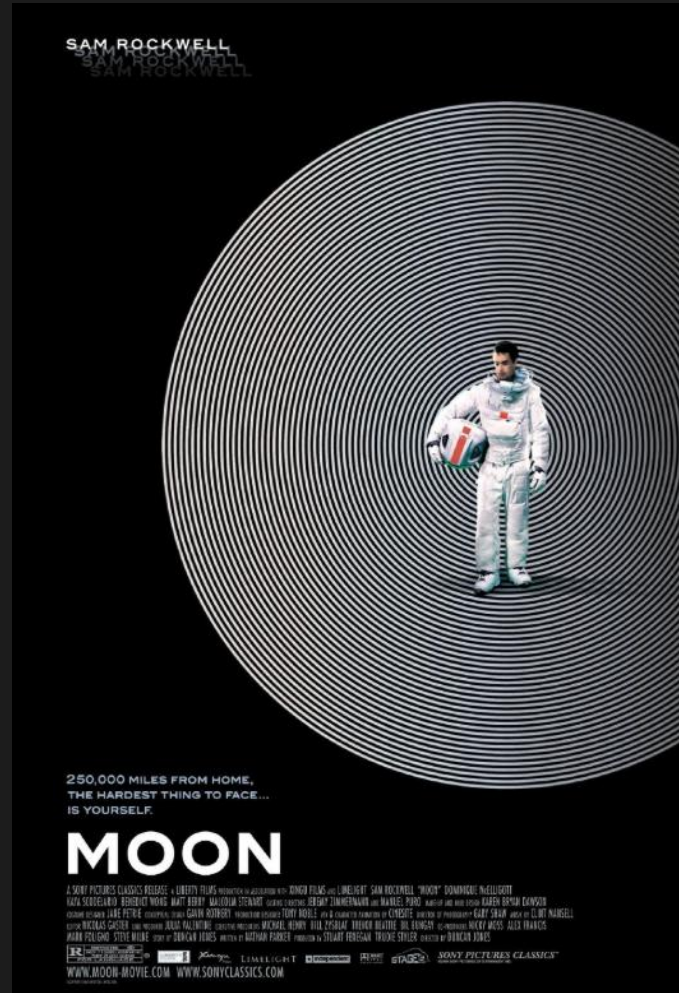
- En av de smarta vakterna ska styras med hjälp av polling och den andra med hjälp av events.
- En av de dumma vakterna ska styras med hjälp av polling och den andra med hjälp av events.
- Vi kommer alltså ha fyra olika controllers:
 - SmartGuardPollController
 - SmartGuardEventController
 - StupidGuardPollController
 - StupidGuardEventController
- De som använder polling ska göra det via en AIPollingStation.
- De som använder events ska göra det via en AEventManager.

UPPGIFTEN

- Alla vakter ska starta på olika positioner, men när vakter med samma beteende överlappar så kommer de troligen inte gå ifrån varandra igen eftersom de har samma beteende.
 - Detta är ok, och t.o.m. bra. Då kan vi verifiera att vi kan uppnå precis samma beteende med olika metoder.
- Sprites för spelare, vakter och datorer finns på Omniway, men det går såklart bra att använda egen art.
- Endast spelare, vakter och datorer behöver visualiseras. Frivilligt att rita ut steering force och desired velocity.
- (reminder to self: visa exempel)

DAGENS AI-FILMTIPS

- Moon (2009), Duncan Jones



FRÅGOR?

